Relational Database Systems

Write the symbolic form first · keep units · check assumptions

INFS2200

DISK GEOMETRY & CAPACITY

(Use consistent byte units)
$s = 2p$ surfaces for p double-sided platters
$C_{\text{track}} = n_b B$
$C_{\text{cyl}} = sC_{\text{track}}$
$C_{\text{disk}} = n_t C_{\text{cyl}}$
Here n_b is blocks/track, B block bytes and n_t tracks/surface

DISK TIMING

$t_{\text{rev}} = 60\frac{000}{\text{rpm}}$ ms
$t_{\text{rot,avg}} = \frac{t_{\text{rev}}}{2}$
$r_{\text{transfer}} = \frac{C_{\text{track}}}{t_{\text{rev}}}$ bytes/ms
$t_{\text{block}} = \frac{B}{r_{\text{transfer}}} = \frac{t_{\text{rev}}}{n_b}$
$t_{\text{access}} = t_{\text{seek}} + t_{\text{rot}} + t_{\text{transfer}}$
$T_{\text{random}}(k) \approx k(t_{\text{seek}} + t_{\text{rot,avg}} + t_{\text{block}})$
$T_{\text{seq}}(k) \approx t_{\text{seek}} + t_{\text{rot,avg}} + kt_{\text{block}}$

RECORDS & BLOCKS

$\text{bfr} = \left\lfloor \frac{B}{R} \right\rfloor$ records/block for fixed unspanned rows
$b = \left\lceil \frac{r}{\text{bfr}} \right\rceil$ blocks
Spanned lower bound: $b = \lceil r\frac{R}{B} \rceil$
R is record bytes and r is record count

INDEX STORAGE

$E = K + P$ bytes/index entry
$f = \left\lfloor \frac{B}{E} \right\rfloor$ entries/block
$b_I = \left\lceil \frac{b_D}{f} \right\rceil$ index blocks
Clustering index $n_I = V(A, R)$ for one entry per ordered value
Dense secondary $n_I = r$ when every record has an entry
Bitmap one vector/value uses $\lceil \frac{r}{8} \rceil$ bytes
Fixed bitmap records: $b_{\text{bitmap}} = \sum_A V(A, R) \left\lceil \frac{K_A + \lceil \frac{r}{8} \rceil}{B} \right\rceil$

B+ TREE SIZING

$p_{\max} = \left\lfloor \frac{B+K}{P+K} \right\rfloor$ internal pointers
$f \approx \lfloor up_{\max} \rfloor$ at fill ratio u
$l_{\max} = \left\lfloor \frac{B-P}{K+P_R} \right\rfloor$ leaf entries
$L = \left\lceil \frac{r}{ul_{\max}} \right\rceil$ leaf nodes
$h \approx 1 + \lceil \log_f L \rceil$ levels (check whether the root already covers all leaves)
Point I/O $\approx h$ plus data access; ranges then follow linked leaves

SELECTIVITY & CARDINALITY

$\text{sel}(c) = \frac{|\sigma_{c(R)}|}{|R|}$
$|\sigma_{c(R)}| \approx |R| \text{ sel}(c)$
Uniform equality: $\text{sel}(A = v) \approx \frac{1}{V(A,R)}$
Key equality: $\text{sel} \approx \frac{1}{|R|}$
Independent AND: $\text{sel}(c_1 \wedge c_2) \approx \text{sel}(c_1)\ \text{sel}(c_2)$
Disjoint OR: $\text{sel}(c_1 \vee c_2) \approx \text{sel}(c_1) + \text{sel}(c_2)$

SELECTION I/O COST

$C_{\text{scan}} = b_R$
Ordered binary search $\approx \lceil \log_2 b_R \rceil$, then add result blocks
Clustered B+ tree $\approx h + b_{\text{match}}$
Unclustered B+ tree $\approx h + n_{\text{match}}$ in the pessimistic one-record-per-block case
Choose the lowest cost using the assumptions given in the question

NESTED-LOOP JOINS

$C_{\text{tuple}} = b_O + n_O b_I$
With M memory blocks: $C_{\text{block}} = b_O + \lceil \frac{b_O}{M-2} \rceil b_I$
One outer block at a time: $b_O + b_O b_I$
Test both orientations; the smaller tuple count helps tuple loops, while the smaller block count usually helps block loops

RELATIONAL ALGEBRA

$\sigma_{c(R)}$ rows · $\pi_{A(R)}$ columns · $\rho(R)$ rename
$R \bowtie S = \sigma_{c(R \times S)}$
$R \overset{c}{\div} S$ returns values related to every tuple in S
For "none": build the forbidden set, then subtract it from the candidate set
Keep join attributes until after the join

QUERY-TREE REWRITE

(1) split conjunctive selections
(2) push selections down
(3) replace $\times$ + join condition with $\bowtie$
(4) perform the most restrictive join first
(5) push projections down while retaining join/output fields
Then compare physical access costs; fewer logical operators does not imply less I/O

DEPENDENCIES & NORMAL FORMS

X^+: start with X, repeatedly add RHS attributes whose LHS is contained in the closure
X is a superkey iff X^+ contains every relation attribute
3NF for every non-trivial $X \to A$, X is a superkey or A is prime
BCNF for every non-trivial $X \to A$, X is a superkey

SCHEDULE TESTS

(1) preserve each transaction's operation order
(2) conflict only across transactions, on the same item, with at least one write
(3) add precedence edge $T_i \to T_j$ when T_i's conflicting operation occurs first
Acyclic graph $\leftrightarrow$ conflict-serialisable; a topological order is the serial order
If T_j reads from T_i: recoverable requires $c_i < c_j$; cascadeless requires $c_i < r_{j(X)}$; strict blocks reads/writes of X until T_i ends

LOCKS & DEADLOCK

(S,S) compatible; any pair containing X is blocked
2PL acquire during growing phase; after first release, acquire no more locks
Wait-for edge $T_i \to T_j$ means T_i waits for T_j; a cycle means deadlock
Basic 2PL gives conflict serialisability, not deadlock freedom; conservative 2PL avoids deadlock

TIMESTAMP ORDERING

$\text{TS}(T)$ is transaction time; each item stores $\text{RT}(X), \text{WT}(X)$
read(X) abort if $\text{TS}(T) < \text{WT}(X)$; else $\text{RT}(X) = \max(\text{RT}(X), \text{TS}(T))$
write(X) abort if $\text{TS}(T) < \text{RT}(X)$ or $\text{TS}(T) < \text{WT}(X)$; else write and set $\text{WT}(X) = \text{TS}(T)$
Transactions never wait, so deadlock is avoided; restarts may starve

RECOVERY

(WAL) write the update log before its data page; force the commit log before reporting success
Steal uncommitted pages may reach disk → UNDO
No-force committed pages may remain in memory → REDO
steal/no-force: UNDO + REDO · steal/force: UNDO · no-steal/no-force: REDO · no-steal/force: neither
After crash: find checkpoint, classify winners/losers, UNDO losers backwards, REDO winners forwards

FAST EXAM CHECKS

- Floor for records per block; ceiling for blocks/nodes
- Keep records, blocks, bytes and I/O distinct
- State uniformity, clustering, fill ratio and buffer assumptions
- Precedence graph tests serialisability; wait-for graph tests live deadlock
- Selectivity is a fraction; cardinality is a count
- `CHECK` handles one-row invariants; multi-row totals need a transaction-safe trigger